Managing Docker Container Lifecycle

In this train, we'll understand how to start, stop, and remove containers, and manage the lifecycle of Docker containers effectively. We'll cover essential commands and best practices for container lifecycle management.

Learning Objectives

By the end of this train you should be able to:

- Understand how to start, stop, remove containers and manage the lifecycle of Docker containers effectively.

Understanding the Docker Container Lifecycle

A Docker container goes through several key states: [source] (https://medium.com/@sangeetv09/docker-container-lifecycle-management-f3575e31fc59).

- **Created:** The container is defined but not started.
- **Running:** The container is actively executing processes.
- **Paused:** The container's processes are temporarily stopped but can be resumed.
- **Stopped:** The container has been shut down but still exists.
- **Deleted/Killed:** The container is removed from the system.

Managing Containers With Essential Commands

1. Creating a Container

```
docker create --name my_container nginx
```

> **Explanation:**
> - `docker create` → Creates a container but does not start it.
> - `--name my_container` → Assigns a name to the container.
> - `nginx` → Specifies the image to create the container from.

2. Running a Container ✖

```
docker run -d --name my_container nginx
```

> **Explanation:**
> - `docker run` → Combines `docker create` and `docker start`.
> - `-d` → Runs the container in detached mode.
> - `nginx` → Uses the `nginx` image to create the container.

- **If a container is already created but stopped, start it using:**

```
docker start my_container
```

3. Listing Running and Stopped Containers ✖

- **View running containers:**

```
docker ps
```

- **View all containers (including stopped ones):**

```
docker ps -a
```

4. Pausing and Resuming Containers ✖

```
docker pause my_container
```

```
docker unpause my_container
```

Explanation:

- `docker pause` sends a SIGSTOP signal, halting all processes.
- `docker unpause` sends a SIGCONT signal, resuming execution.

5. Stopping and Restarting Containers ✖

```
docker stop my_container
```

```
docker restart my_container
```

Explanation:

- `docker stop` sends a SIGTERM signal for a graceful shutdown.
- `docker restart` stops and starts the container again.

6. Killing and Removing Containers ✖

```
docker kill my_container
```

```
docker rm my_container
```

```
docker container prune
```

Explanation:

- `docker kill` sends a SIGKILL signal, stopping the container immediately.
- `docker rm` deletes a container from the system.
- `docker container prune` removes all stopped containers to free up space.

Conclusion

Managing the Docker container lifecycle ensures smooth application deployment and efficient resource utilization. By mastering these commands, you can control how containers run, stop, restart, and clean up after use.

Key Takeaways ✖

- **Takeaway 1:** Docker containers move through different states: Created → Running → Paused → Stopped → Deleted.